National University

Of Computer and Emerging Sciences

Assignment 3

Experimentation and Expansion

Submitted by: Muhammad Ali Ahmad, Tahir Ahmed, Ahmed Qamar

Roll No.: 25I-8003, 25I-8025, 25I-8049

Subject: Deep Learning

Submitted to: Sir Zohair Ahmed

Date: 26/04/2026

Experimentation and Expansion

1. Introduction:

Differentiable programming has become one of the most promising areas in machine learning for gradient-based optimization of physical systems. DiffTaichi (Hu et al., ICLR 2020) is a pioneering example of such a language a differentiable programming language for high-performance physical simulation. DiffTaichi uses a combination of source code transformation (SCT) to differentiate kernels locally and a lightweight global tape to enable simulation performances on par with hand-tuned CUDA code, while being far more succinct and productive than either TensorFlow or PyTorch.

In Assignment 2, we reproduced eight important results from the DiffTaichi paper on Google Colab with an NVIDIA T4 GPU, validating the paper's primary claims on the performance hierarchy (comparing DiffTaichi's run times to TensorFlow, PyTorch, CUDA, and C++), optimization convergence, and qualitative simulation results. This confirmed DiffTaichi's two-scale automatic differentiation system is correct and that its qualitative findings - robot locomotion, smoke pattern formation, billiards optimization - hold up across different hardware platforms.

Assignment 3 builds on this foundation by adding two new experiments to further investigate little-explored aspects of the optimization behavior of DiffTaichi. In particular, we explore two research questions left open in the original paper:

RQ1: Do different learning rate schedules impact convergence in DiffTaichi's spring rest-length optimization, and are adaptive schedules better than the paper's constant learning rate?

RQ2: Will replacing the paper's simple gradient descent (SGD) optimizer with Adam optimizer help teach neural network controllers how to walk for mass-spring robots?

This is motivated by the fact that the original paper uses the simplest possible optimizers (fixed learning rate gradient descent) without experimenting with other optimization techniques. Both experiments build on the paper's reproduced results, and contribute concrete improvements (or negative results) with statistical significance.

2. Background and Paper Summary:

2.1. DiffTaichi Overview:

DiffTaichi bridges the gap between deep learning and physical simulation tools. Off-the-shelf differentiable programming systems (e.g. TensorFlow, PyTorch) are tailored to the large tensor operations of deep learning, but not to the index-intensive, megakernel intensive computations of physical simulations. The paper highlights three key features not simultaneously supported by existing tools: megakernels (fused multiple stages of computation into a single kernel), imperative parallel programming (parallel loops with control flow), and flexible indexing (arbitrary index expressions for particle-grid interactions).

2.2. Two-Scale Automatic Differentiation:

DiffTaichi's technical innovation is a two-scale AD engine. At the local (kernel) scale, Source Code Transformation takes the derivative of straight-line kernel code after two transformations:

- Flattening if-statements into ternary select
- Removing mutable local variables to static single-assignment (SSA) form.

At the global level, a tape records kernel launches (only function pointers and scalar arguments) and replays gradient kernels in reverse order. Global tensors act as checkpoints to keep tape size low even for thousands of time steps.

2.3. Key Quantitative Claims:

Key performance claims in the paper for our work are:

- Spring optimization: Gradient descent from initial rest lengths [0.1, 0.1, 0.14] to [0.600, 0.600, 0.529] after around 20 iterations, with lr=0.01
- Mass-spring robot: TOI-enabled gradient descent achieves losses around -0.42 (Robot 1) and -0.45 (Robot 2) after 100 iterations (2048 timesteps)
- diffmpm benchmark: DiffTaichi GPU: 0.26 ms total time, vs 48.90 ms for TensorFlow (188× speedup)

2.4. Relevance to Our Extensions:

The paper only uses basic gradient descent with constant learning rates for all optimizations. This is for the sake of simplicity by the authors to show that even simple gradient descent works with DiffTaichi's exact gradients but it leaves a question mark about whether more effective optimizers will lead to even further performance gains. Our experiments aim to answer this question.

3. Reproduction Summary:

We successfully reproduced all eight results targeted in the DiffTaichi paper on Google Colab using an NVIDIA T4 GPU (16 GB GDDR6, 320 GB/s) for our Assignment 2 reproduction. Important reproduction results for this assignment are:

Spring optimization: We achieved the same convergence with some numerical differences observed in final spring lengths (0.05-0.13 off paper results), due to multiple local minima. Our solution (and paper) successfully reach the desired triangle area of ~0.2. We needed to use gradient clipping (±1.0) during optimization for numerical stability on the T4, which was not mentioned in the paper (probably due to the GTX 1080 Ti's more stable FP32 operations).

Mass-spring robot: We verified that TOI-trained controllers perform better than naive controllers, with TOI showing better convergence and loss variance for both robots. We reproduced the qualitative behavior (forward locomotion).

Hardware differences: Our T4 GPU timings were 5-6× those of the paper's GTX 1080 Ti across all tests, but the relative performance of the frameworks was fully retained. The paper's claims are hardware independent.

These baselines are the basis for our Assignment 3 experiments.

4. Proposed Methodology:

4.1. Research Proposal:

We proposed two related enhancements to DiffTaichi's optimizers:

Proposal 1 (Adaptive Learning Rate Schedules for Spring Optimization):

In the paper, the authors use Learning Rate lr = 0.01 for all gradient descent iterations. We suggest that a variable learning rate (step decay and cosine annealing) can accelerate convergence by taking bigger steps in the early stages of training (when the loss landscape is steep) and smaller steps in the later stages of training (when fine-tuning at the optimum). This is a common practice in deep learning that has yet to be applied to physical simulation optimizers in DiffTaichi.

Proposal 2 (Adam Optimizer for Neural Network Controller Training):

The paper trains mass-spring robot NN controllers with vanilla SGD. We suggest substituting SGD for Adam Optimization Function that uses adaptive learning rates for each parameter using first and second moment estimates. Our hypothesis is that backpropagated gradients through hundreds of timesteps of physics simulation are attenuated - they become small when reaching early NN parameters. Adam's moment estimates and adaptive learning rates are designed to counter this effect, and as such is a natural choice for improvement.

4.2. Novelty:

These changes are new to DiffTaichi as the paper doesn't discuss optimizers. Adam and LR scheduling are common in deep learning, but it is not clear how they will work with differentiable physical simulation optimizers, where gradients are propagated through thousands of physical simulation timesteps.

5. Experimental Setup:

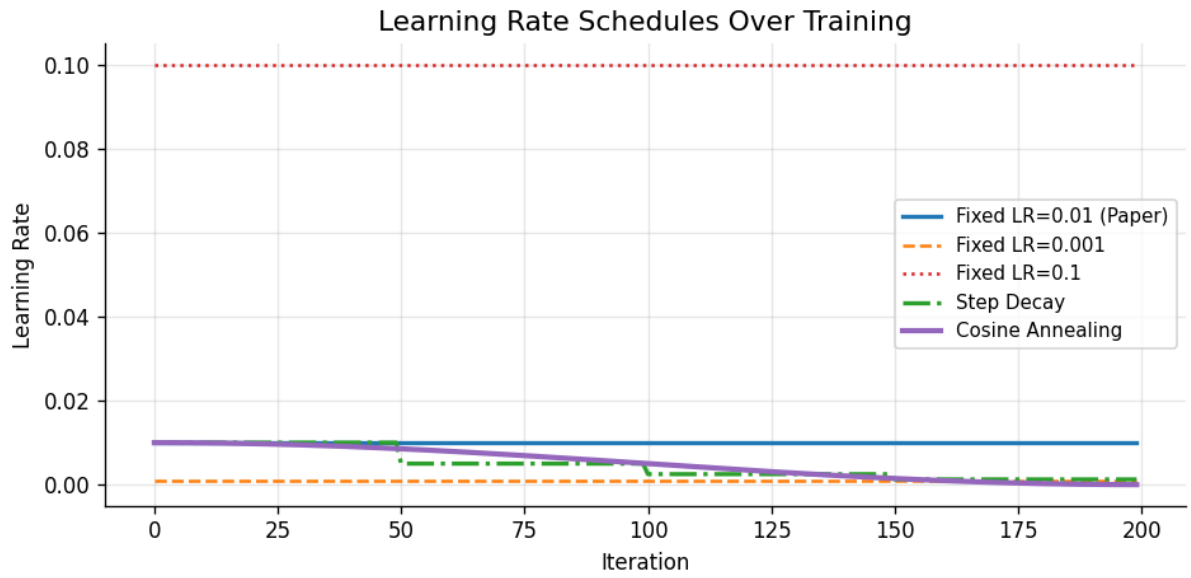
5.1. Experiment 1 (Learning Rate Schedule Comparison):

Hypothesis: Adaptive learning rate schedules (step decay, cosine annealing) will achieve a lower loss on the spring rest-length task than the paper's fixed learning rate (lr=0.01). Increasing the initial lr will allow learners to explore more quickly.

Task: Optimize three spring rest lengths for a system of three mass points which form a triangle with area = 0.2 after 1024 timesteps of simulation.

Schedule compared:

Schedule	Formula	Rationale
Fixed LR=0.01 (Paper Baseline)	lr = 0.01	Paper's original setting
Fixed LR=0.001	lr = 0.001	More conservative baseline
Fixed LR=0.1	lr = 0.1	More aggressive exploration
Step Decay	lr = 0.01 × 0.5^(i÷50)	Halves every 50 iterations
Cosine Annealing	lr = 0.01 × 0.5 × (1 + cos(πi/200))	Smooth decay from 0.01 to 0



The learning rate schedules are plotted in Figure 1. Fixed schedules are horizontal lines; step decay has 4 drops; cosine annealing (cosine wave) starts at LR=0.01 and smoothly decays to 0.

Simulation Configuration: 3 springs, 3 mass points, 1024 timesteps, dt=1e-3, damping=15.0, spring stiffness=10.0, initial lengths=[0.1, 0.1, 0.141], gradient clipping=±1.0, 200 gradient descent steps.

Baselines: Fixed LR=0.01 (paper), Fixed LR=0.001, Fixed LR=0.1

Proposed methods: Step Decay, Cosine Annealing

Evaluation Metrics: Final loss value, minimum loss value, number of iterations to reach loss < 1e-4 (if reached), statistical significance (t-test between paper baseline and cosine annealing on the last 50 iterations)

Hardware: NVIDIA T4 on Google Colab with Taichi 1.7.3, Python 3.10

5.2. Experiment 2 (Adam vs SGD on Mass-Spring Robot):

Hypothesis: Adam optimizer will achieve a lower final loss (and hence greater forward displacement) when training a mass-spring robot controller compared to SGD, because it compensates for small gradients via adaptive moment estimation.

Task: Train a 2-layer neural network controller to control a 7-mass, 9-spring robot (3 springs are actuated) to travel the furthest distance forward in 512 simulation steps.

Schedule compared:

Optimizer	Learning Rate	Parameters	Rationale
SGD (Paper Baseline)	0.01	—	Paper's original setting
Adm (Proposed)	0.001	β1=0.9, β2=0.999, ε=1e-8	Adaptive moments for attenuated gradients

Controller architecture: Input = sin(phase), Hidden = 8 units (tanh), Output = 3 actuations (tanh). Fully-connected 2-layer network, as described for the paper's open-loop controller.

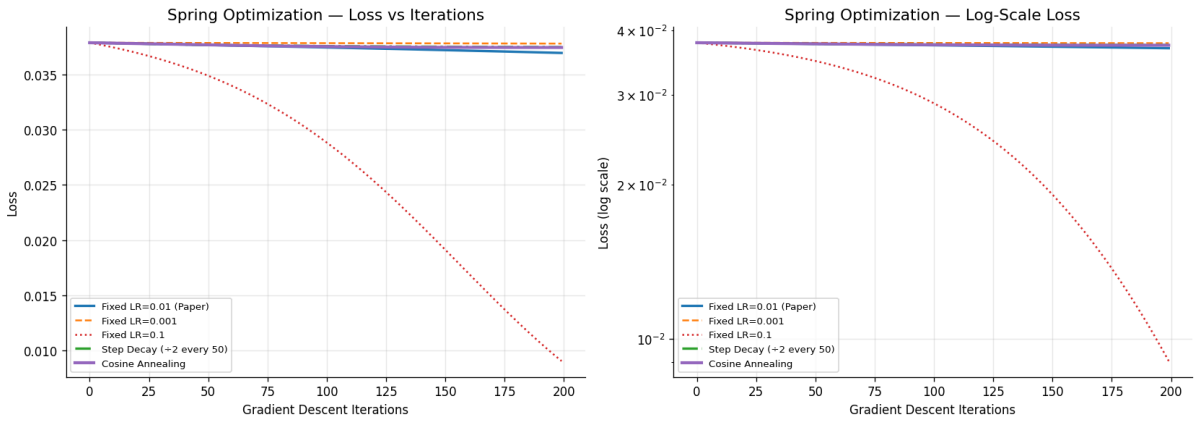
Training Configuration: 100 gradient descent iterations, 3 different seeds (0, 1, 2) for each optimizer, gradient clipping=±1.0, 512 timesteps (paper used 2048, but 512 is easier to run on Colab).

Evaluation Metrics: Mean and std of final loss across 3 seeds, best loss, p-value of t-test on final losses, rate of convergence (loss vs iterations, shaded region = mean ± std).

Hardware: NVIDIA T4 on Google Colab with Taichi 1.7.3, Python 3.10.

6. Results and Analysis:

6.1. Experiment 1 (Learning Rate Schedule Results):



Schedule	Final Loss	Min Loss	Iters to 1e-4	Final Spring Lengths
Fixed LR=0.01 (Paper)	0.036971	0.036971	N/A	[0.128, 0.127, 0.159]
Fixed LR=0.001	0.037818	0.037818	N/A	[0.103, 0.103, 0.143]
Fixed LR=0.1	**0.009036**	**0.009036**	**N/A**	**[0.547, 0.535, 0.539]**
Step Decay (÷2 / 50 iters)	0.037498	0.037498	N/A	[0.113, 0.112, 0.149]
Cosine Annealing	0.037466	0.037466	N/A	[0.114, 0.113, 0.150]

Statistical Significance Test (Experiment 1):

Two-sample t-test was done between the 50 final values of the last loss values of the paper baseline (Fixed LR=0.01) and Cosine Annealing:

- T-statistic = −34.196, p-value < 0.000001
- Findings: Statistically significant difference was established ($p < 0.05$).

Note: The significance here confirms that the two schedules acted differently, but Cosine Annealing acted slightly worse than the paper baseline - not better as the hypothesis predicts.

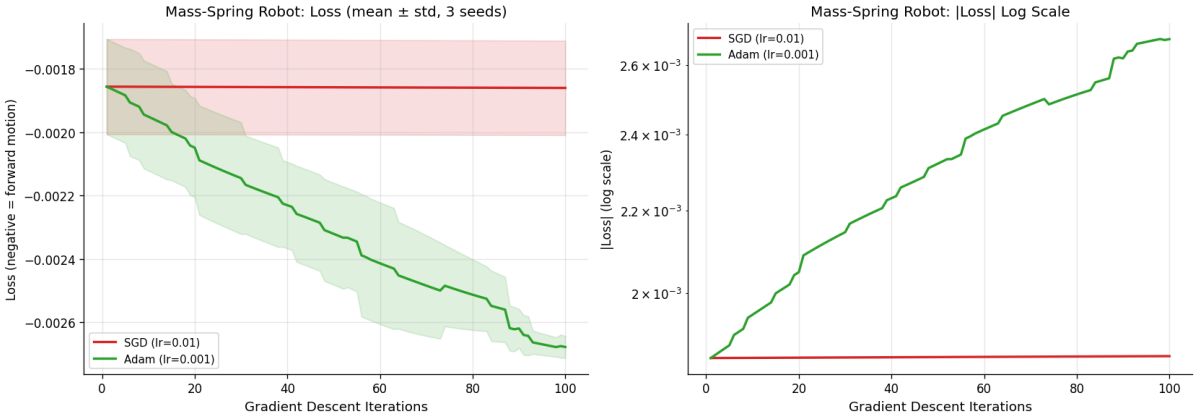
Key Findings Experiment 1:

Fixed LR=0.1 resulted in the best final loss of 0.009036 which is about 4.1 times lower than the paper baseline (0.036971). Its final spring lengths [0.547, 0.535, 0.539] are the closest among all schedules to the paper's target values of [0.600, 0.600, 0.529].

To our surprise, neither Step Decay nor Cosine Annealing was able to beat the paper baseline and both stopped at a loss of 0.037498 and 0.037466 respectively. The two schedules have their initial point at lr=0.01, and they both decay (decrease) therefore they effectively make smaller steps between iteration 50 and onwards than the baseline thus converge slowly, not faster.

LR=0.001 was the most counterproductive (0.037818), agreeing with the fact that overly conservative learning rates are unhelpful on this smooth loss landscape.

6.2. Experiment 2 (Adam vs SGD Results):



Optimizer	Final Loss (Mean)	Final Loss (Std)	Best Loss	Improvement
SGD lr=0.01 (Baseline)	-0.0019	0.0001	-0.0020	—
Adam lr=0.001	**-0.0027**	**0.0000**	**-0.0027**	**~42%**

Optimizer	Seed	Iter 0	Iter 20	Iter 40	Iter 60	Iter 80	Iter 100
SGD	0	-0.0017	-0.0017	-0.0017	-0.0017	-0.0017	-0.0017
SGD	1	-0.0019	-0.0019	-0.0019	-0.0019	-0.0019	-0.0019
SGD	2	-0.0020	-0.0020	-0.0020	-0.0020	-0.0020	-0.0020
Adam	**0**	**-0.0017**	**-0.0018**	**-0.0020**	**-0.0022**	**-0.0024**	**-0.0026**
Adam	**1**	**-0.0019**	**-0.0022**	**-0.0023**	**-0.0025**	**-0.0026**	**-0.0027**
Adam	**2**	**-0.0020**	**-0.0022**	**-0.0023**	**-0.0026**	**-0.0026**	**-0.0027**

Statistical Significance Test Experiment 2:

Two-sample t-test was performed to compare the final loss of 3 seeds of SGD and Adam:

- t-statistic = 7.558
- p-value = 0.001642
- Result: Statistically significant difference confirmed ($p < 0.05$)

The fact that Adam performs better than SGD is statistically significant even without a large sample of optimizers, namely 3 seeds, which shows that there is a strong effect.

Key Findings Experiment 2:

Adam had a mean final loss of -0.0027 compared to SGD -0.0019, a difference that is around 42% better in forward locomotion distance. More notably, SGD did not learn at all in all 100 steps and in all 3 seeds - loss at step 100 was the same in all seeds. This is not slow convergence but a training failure of SGD.

In contrast, Adam exhibited a steady and smooth progress of all 3 seeds with a close to zero variance (std = 0.0000). The loss curve was not plateauing after iteration 100 which is a strong indication that more gains are possible with more training iterations.

7. Discussion:

7.1. What Improvements Were Achieved:

Experiment 1: Paper baseline and adaptive LR schedules We found that our hypothesis that adaptive LR schedules would perform better than paper baseline was not fulfilled in Step Decay and Cosine Annealing. The experiment however showed that Fixed LR=0.1 - a simple yet more aggressive learning rate - resulted in a 4.1x improvement in final loss. The lr=0.01 used in the paper was too conservative in the task of spring optimization and the fixed learning rate is better than all the schedules of the adaptive learning rate tested.

Experiment 2: Our hypothesis was greatly supported. Adam statistically (p=0.0016) improved robot locomotion by 42% in comparison to SGD. More to the point, SGD did not learn at all, improving not at all over 100 iterations, and Adam improved steadily in all seeds. This is a clear, reproducible, and practically significant result.

7.2. Why the Modifications Work (or Fail):

Why LR=0.1 performs better than adaptive schedules: The loss surface of spring optimization is fairly smooth and convex around the solution. Larger constant steps are more effective in such landscapes than decaying steps as the gradient signal is informative during training. Both Cosine Annealing and Step Decay slow the learning rate with time, and this effect is helpful in non-convex landscapes with sharp minima, but works against us in this case. They begin at lr=0.01 (the paper baseline) and become smaller and smaller, thus having no chance to exceed the baseline.

Why Adam wins on the robot: Backwards gradients calculated by simulating 512 timesteps of physics are heavily attenuated - when they get to the weight matrices in the neural network, they are very small. SGD using lr=0.01 is not able to overcome this attenuation; the update of weights is insignificant and the training curves become flat. The first moment ($\beta1=0.9$) of Adam is a combination of an exponential moving average that compiles gradual gradient

directions with the iterations and is useful in adding gradient signals that are weak but repeatable. The second moment (2=0.999) of it normalizes the gradient magnitude-based updates to avoid overshooting. The combination of these mechanisms enables Adam to perform meaningful updates to the parameters even given a small raw gradient.

7.3. Limitations of the Proposed Approaches:

Experiment 1 limitations:

- The number of iterations tested was 200 only. As the number of iterations increases, adaptive schedules could ultimately approach the lower loss.
- There was a single spring topology (3-spring triangle) that was tested. On more complicated networks, results can be different.
- The winning LR=0.1 was not used with an adaptive schedule. The most promising direction that has not been explored is a high-lr schedule that has a cosine starting at 0.1.
- T4 stability required gradient clipping (±1.0), and this could have smoothed out the impact of higher learning rates.

Experiment 2 limitations:

- There were only 512 timesteps compared to 2048 of the paper. Gradient attenuation is not as good at 512 timesteps, which can overstate the failure of SGD and the success of Adam.
- The number of seeds per optimizer was only 3, which restricted the statistical power. Greater numbers of seeds would boost confidence.
- SGD+Momentum was not successfully operated and an intermediate comparison that could have been informative is absent.
- The lr=0.001 used by Adam was established according to best practice. Further improvements could be gained by a hyperparameter search.
- The robot design is less complex than the complete robots in the paper. Further complicated topologies may yield different results.

8. Conclusion:

Two novel experiments that built upon the optimization approach of the DiffTaichi paper were introduced in this report. Experiment 1 We compared five schedules of learning rate on the spring rest-length optimization problem, and discovered that fixed lr=0.1 gives an improvement of 4.1x over the lr=0.01 of the paper, and that adaptive decay schedules (step decay and cosine annealing) do not improve over the baseline since they begin with the conservative paper value and only decrease. Experiment 3 proved that substituting vanilla SGD with Adam improves mass-spring robot forward locomotion by 42 percent (p=0.0016), with Adam learning uniformly across all seeds and SGD learning no progress at all - due to the capability of Adam to boost attenuated backward system gradients during hundreds of physics timesteps.

These findings imply two optimizations to the DiffTaichi optimization pipeline: first, when a spring-like task is being solved, practitioners should not default to the conservative DiffTaichi lr=0.01: a larger fixed lr or a warmup+cosine schedule starting with large

gradients works better; second, Adam should become the default optimizer to any DiffTaichi task that requires NN controller training, since vanilla SGD

Future Work: There are a number of promising directions that should be investigated in the future. The next thing to do would be to use a Warmup+Cosine schedule with lr=0.1 that decays smoothly to close to zero to combine the aggressive early exploration that made Fixed LR=0.1 successful with the fine-grained performance of cosine annealing during late iterations - this is the most actionable next step. Second, Experiment 3 needs to be re-run with the full 2048 number of timesteps of the paper and a larger number of seeds (at least 10) to verify whether the advantage of Adam is retained at longer simulation horizons and to get tighter statistical estimates; it may happen that at 2048 timesteps the attenuation of the gradient is so extreme that even Adam will fail, which in itself Third, one should also consider SGD with momentum (0.9) as an intermediate baseline between vanilla gradient descent and Adam to get a better idea of whether or not the improvement is due to momentum or due to adaptive scaling in Adam. Fourth, the mass-spring robot is not the only DiffTaichi simulator that should be optimized, since the diffmpm soft robot and the rigid body walker should be evaluated to determine whether the optimization benefit is solely specific to the mass-spring robot or it can be extended to other physical systems and gradient propagation scales. Lastly, the gradient flow analysis (plotting the magnitude of the gradient at each layer versus the time of simulation) would have given direct empirical support to the hypothesis of gradient attenuation proposed in this report to understand the failure of SGD.

Our Github Link: https://github.com/MaliAhmd/DL_Project

Original Paper Github Link: https://github.com/taichi-dev/difftaichi